FINAL SUBMISSION · SS26 SWEN2 · TEAM 3

TourPlanner

Project Protocol & Technical Documentation

Authors	**Ricky Raveanu** (backend) & **Nikola Petrovic** (frontend)
Course	SS26 SWEN2 – Software Engineering 2
Team	Team 3
Submission	Final (Class 24)
Period	February 3 – June 21, 2026
Repository	GitHub — git history is integral to this documentation
Stack	ASP.NET Core 8 · React 19 + TypeScript · PostgreSQL 18 · Docker · OpenRouteService · Leaflet

> **Note on git history:** The commit history records the full development sequence (Feb–Jun 2026) including author attribution. This protocol covers architecture, UX wireframes, UML diagrams, unit-test rationale, and time tracking for the final submission.

1 Technical Steps and Decisions

1.1 Framework Choice — React (teacher-approved)

> The project specification lists Angular as the default frontend framework. The team requested and received teacher approval to use **React 19 + TypeScript + Vite** instead. The MVVM architecture requirement is fully satisfied: Models are TypeScript interfaces in `src/models/`, ViewModels are custom hooks in `src/viewmodels/`, and Views are React components in `src/views/` that never import from `src/api/` directly.

1.2 Technology Stack

Backend	ASP.NET Core 8, C# — three-layer architecture (API / BL / DAL), EF Core ORM
Frontend	React 19 + TypeScript + Vite — MVVM via TanStack Query & Form, Tailwind CSS + shadcn/ui
Database	PostgreSQL 18, code-first migrations via EF Core
Auth	ASP.NET Identity + JWT access tokens (15 min) + refresh tokens (7 days) in DB
Logging	Serilog — rolling daily log files (`logs/`) + console sink
Map / Routing	OpenRouteService API (geocoding + directions); Leaflet for interactive map rendering
Validation	FluentValidation on the backend; Zod / TanStack Form on the frontend
Testing	NUnit 3 + Moq — 36 unit tests targeting the Business Logic layer
Infra	Docker Compose (dev + prod), self-hosted via Coolify, NGINX reverse proxy

1.3 Architecture Decisions

Three-Layer Backend

Controllers (`TourPlanner.Api`) delegate all logic to BL services. Services (`TourPlanner.BL`) call only DAL repository interfaces. Repositories (`TourPlanner.DAL`) interact only with EF Core. Each layer has its own exception type — `BusinessLogicException`, `DataAccessException` — translated at boundaries so callers only see appropriate abstractions.

Repository Pattern

`ITourRepository`, `ITourLogRepository`, and `IRefreshTokenRepository` abstract all database access. Concrete implementations use EF Core; unit tests inject Moq fakes. No layer ever bypasses the interface to use a concrete repository directly.

JWT + Refresh Token Authentication

Short-lived JWTs are issued on login/register. Refresh tokens (opaque random strings) are stored in the `RefreshTokens` table and rotated on each refresh. The API is fully stateless; sessions are maintained entirely on the client via a Zustand auth store that persists tokens to `localStorage`.

ORS Routing Proxy

OpenRouteService calls are proxied through the ASP.NET backend (`MapController` → `OrsService`) rather than made directly from the browser. This eliminates CORS preflight issues and keeps the ORS API key server-side, never exposed in the JavaScript bundle.

Computed Tour Attributes

`Tour.Popularity` (count of attached logs) and `Tour.ChildFriendliness` (derived from average difficulty, distance, and time across logs) are pure C# computed properties on the domain model — not stored columns. EF Core ignores them via `t.Ignore()` in `OnModelCreating`. Because they cannot be expressed in SQL, the full-text search that must also match them is performed in the BL (see §4.2).

Route Persistence at Create Time

When a tour is created — or its from/to/transport type changes — the BL calls OpenRouteService once and stores the computed distance, estimated time and the route geometry with per-point elevation on the tour. The map and elevation chart then render from this stored data instead of re-querying ORS on every view, so the displayed distance, the chart and the drawn route stay consistent. A shared `RouteEnricher` holds this logic and is reused by tour creation and by a startup *backfill* that fills in any tour still missing route data — so a tour created during a transient ORS outage self-heals on the next restart.

Import / Export

Tour data can be exported to and imported from JSON via `ImportExportService`. Export returns all of the user's tours with their logs; import recreates them under the current user with fresh identifiers and resolves name collisions by appending a counter. Malformed input is rejected with HTTP 400 rather than crashing the request.

1.4 Design Patterns

Repository Pattern

All data access abstracted behind `ITourRepository`, `ITourLogRepository`, `IRefreshTokenRepository`. BL never touches EF Core directly.

Options Pattern

`JwtSettings`, `OrsOptions`, `ImageServiceOptions` injected via `IOptions<T>`. Secrets come from environment variables, not source code.

MVVM (Frontend)

React views in `src/views/` consume ViewModel hooks in `src/viewmodels/`. Views never import `src/api/` directly — strictly enforced by ESLint rule.

Middleware Pipeline

`ExceptionHandlingMiddleware` catches domain exceptions and maps them to HTTP status codes (404, 409, 401, 500) before reaching the controller.

Strategy (Transport Type)

`TransportType` enum drives ORS routing profile: `foot-hiking`, `cycling-regular`, `driving-car`.

Generic Reusable Component

`DataTable<TData>` (powered by TanStack Table) is a fully generic, type-safe table component used for both tour list and tour log list.

1.5 Key Challenges and Solutions

CORS + ORS API key exposure

Direct ORS requests from the browser triggered CORS errors and would have exposed the API key in JS. **Solution:** route all map/routing calls through `MapController` on the backend, which injects the key from an environment variable.

Docker dev API proxy (SWEN2-68)

The React dev container could not resolve `api:5000` by hostname. **Solution:** NGINX inside the frontend container proxies `/api/*` to `api:5000`, using Docker Compose service-name DNS.

Leaflet z-index bleeding over dialogs (SWEN2-73)

Leaflet's map tiles rendered on top of modal dialogs (z-index conflict). **Solution:** wrap `MapContainer` in a div with `isolation: isolate` so the map forms its own stacking context.

ORS geocoding returning US results for European cities

Geocoding "Vienna" returned Vienna, Virginia (USA) instead of Austria. **Solution:** added a `focus.point` parameter centred on Central Europe (lat 48, lon 14) to bias results geographically.

Uniqueness validation for tours and logs

Duplicate tour names per user caused silent data collisions. **Solution:** added `ExistsWithNameAsync` to the repository and a guard in `TourService.CreateAsync` that throws `BusinessLogicException`, surfaced as HTTP 409.

ORS coordinate order — routes rendered in Yemen

ORS GeoJSON uses `[lon, lat]` while Leaflet expects `[lat, lon]`. A missed swap placed Austrian routes off the coast of Yemen (with Arabic map tiles). **Solution:** normalise to `[lat, lon]` in the backend before returning the geometry, and store it in that order so every consumer is consistent.

Stale preview deployments (Coolify)

Coolify did not always rebuild the backend image on a force-push, so PR previews served an old API (e.g. a 404 on a newly added endpoint) behind an up-to-date frontend. **Solution:** diagnosed via the deployed Swagger document, then redeployed without build cache; production rebuilds reliably on normal pushes.

2 Application Features – UML Use Case Diagram

All features delivered in the final submission are shown below. Registration and Login are the only unauthenticated entry points. All other use cases require a valid JWT access token.

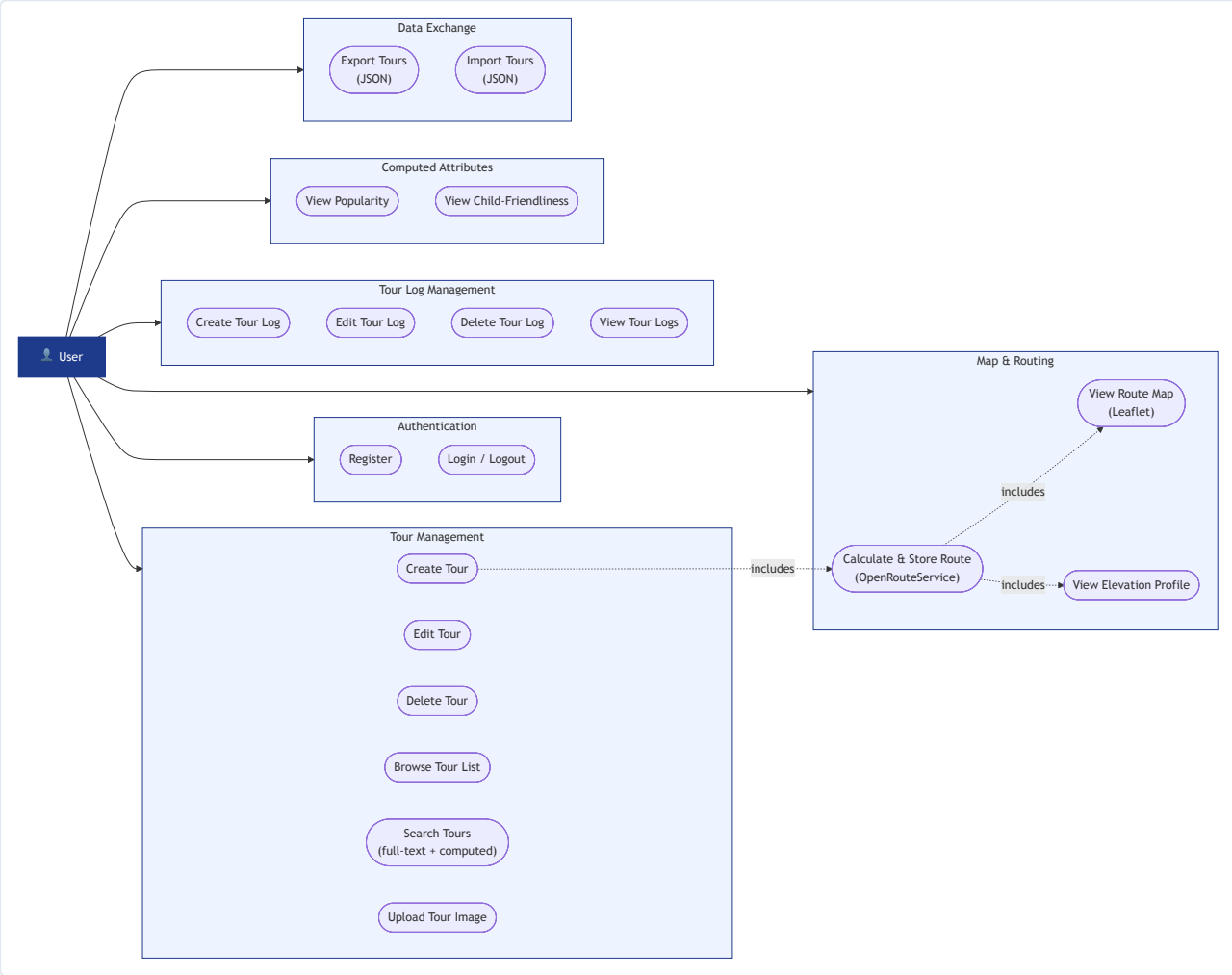


Figure 1 – TourPlanner Use Case Diagram (Mermaid flowchart)

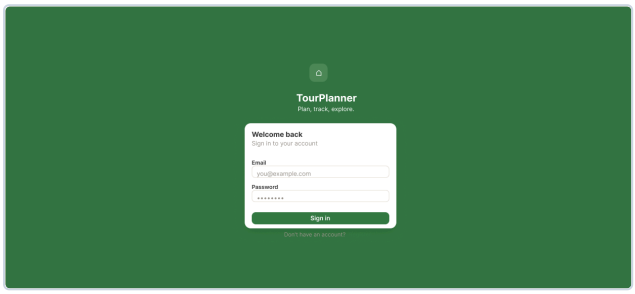
«includes» relationships:

- *Create Tour* «includes» *Calculate Route* — distance and estimated time are always fetched from OpenRouteService when a tour is saved.
- *Calculate Route* «includes» *View Route Map* — the stored GeoJSON geometry is rendered on the Leaflet map in the Tour Detail view.
- *Calculate Route* «includes» *View Elevation Profile* — per-point elevation returned by ORS powers the Recharts area chart (the unique feature).

3 UI Flow – Wireframes

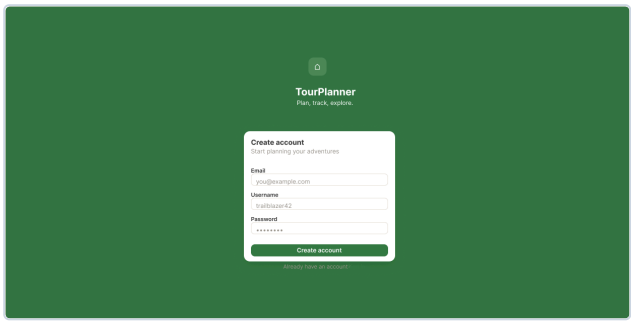
Wireframes were produced before implementation to fix the two-panel layout (tour list left, detail right) and the modal dialog interaction pattern for CRUD operations.

Login



Full-page auth form. Email + password with link to registration.

Register

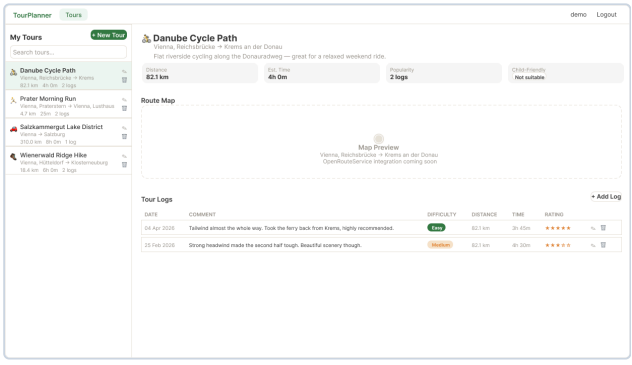


Self-registration with email, username, password, and inline validation.

Tours – Empty State

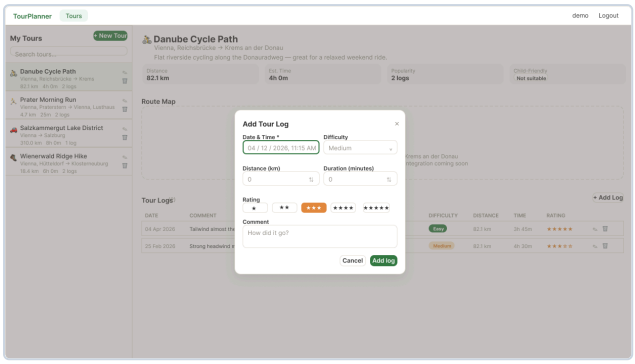
After login with no tours created yet. Prominent create-tour call-to-action.

Tours – Detail with Map



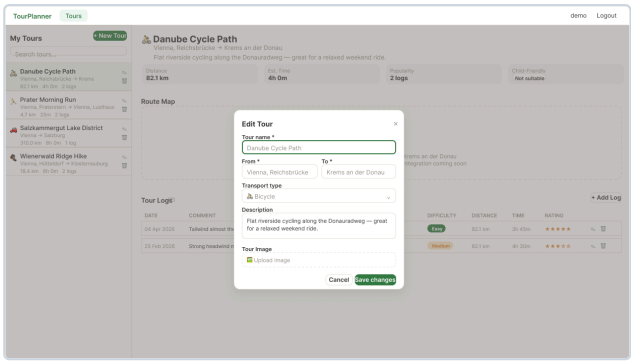
Tour selected: two-panel layout with attributes on the left and Leaflet route map on the right.

Tours – Logs Tab



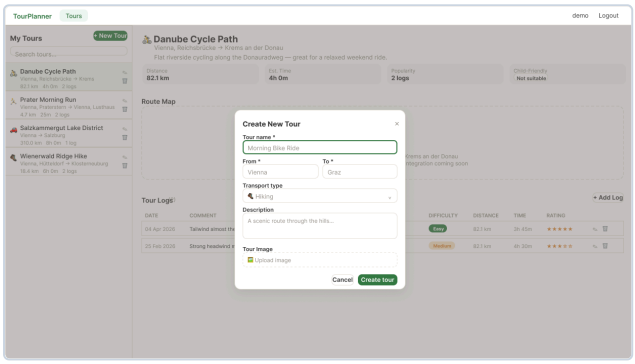
Per-tour log list: date, distance, total time, difficulty, and star rating.

Tours – Edit Dialog



Modal for editing tour name, description, from/to, and transport type.

Tours – Log Create / Edit



Modal for adding or editing a tour log: date, comment, difficulty, distance, time, rating.

Navigation Flow

Login / Register

↓

Tour List (left panel)

↕ select / create / search

Tour Detail + Map (right panel)

↕ Logs tab

Tour Log List

↕ + / edit button

Log Modal Dialog

4 Application Architecture

4.1 Class Diagram

The diagram covers the domain model, repository interfaces, and key BL services. Frontend TypeScript interfaces in `src/models/` mirror the same structure.

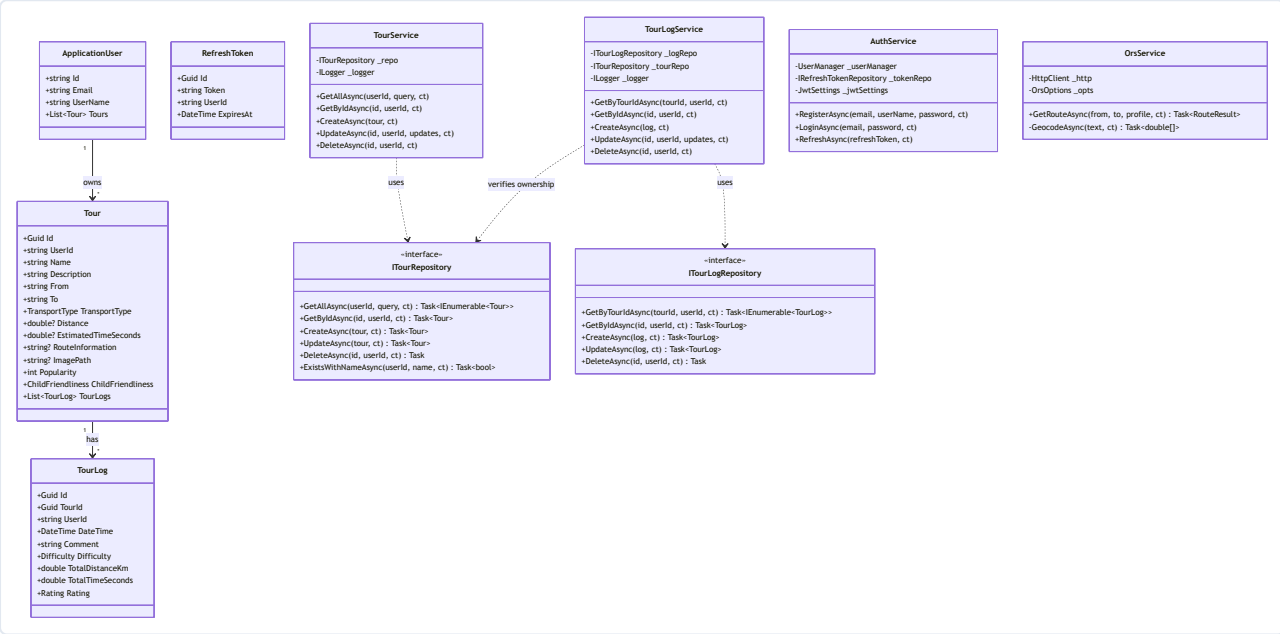


Figure 2 – Class Diagram (Mermaid classDiagram)

> **Layer enforcement:** Services depend only on repository *interfaces*; controllers depend only on service *interfaces*. No layer may skip a level. `Popularity` and `ChildFriendliness` are computed C# properties (not DB columns) — EF Core ignores them via `t.Ignore()`.

4.2 Sequence Diagram – Full-Text Search

Full-text search is triggered by a debounced input in the TourListView. The query travels from the React ViewModel through the HTTP API to the BL, which loads the user's tours from the DAL and filters them in memory — so the search can match both stored fields and the computed Popularity / ChildFriendliness attributes (impossible in SQL).

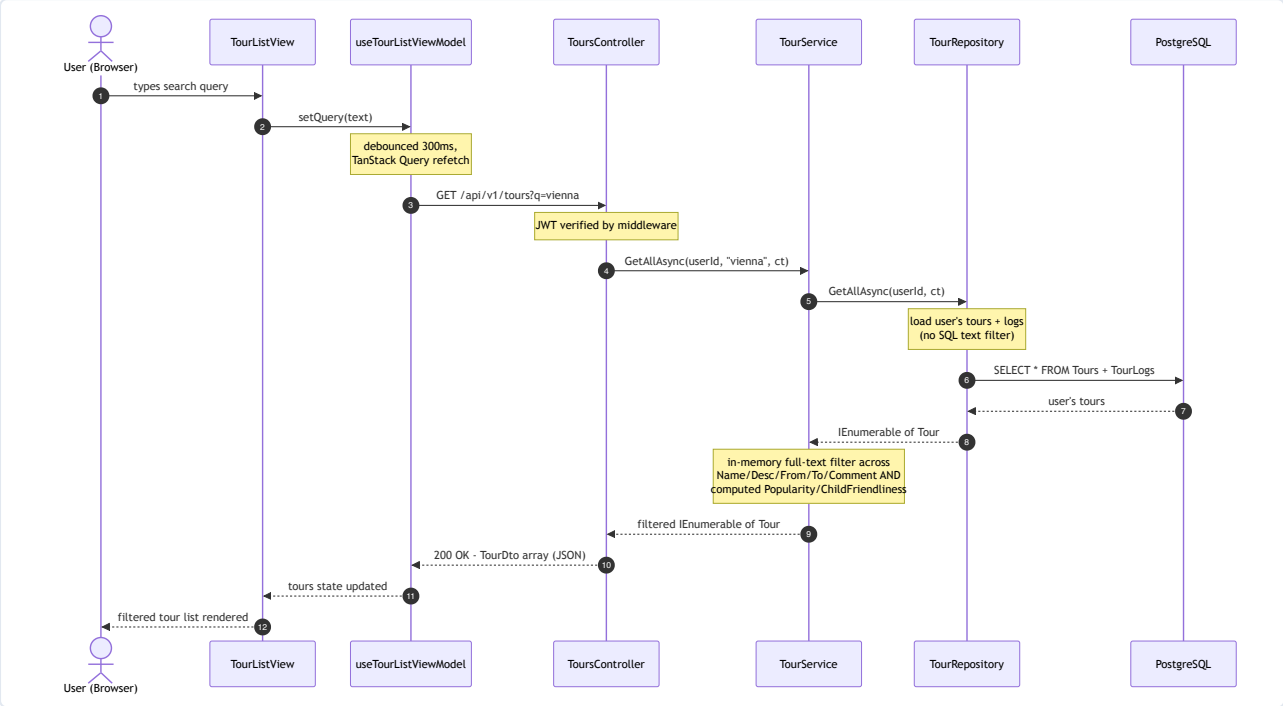


Figure 3 – Full-Text Search Sequence (Mermaid sequenceDiagram)

In-memory full-text filter (TourService.GetAllAsync)

```
var tours = await _tourRepository.GetAllAsync(userId, ct);   // DAL: all of the
user's tours
if (string.IsNullOrWhiteSpace(searchQuery)) return tours;

var q = searchQuery.Trim().ToLowerInvariant();
return tours.Where(t => MatchesSearch(t, q)).ToList();        // BL: text + computed
attributes

// MatchesSearch covers Name/Description/From/To/TransportType, every log Comment,
// the computed Popularity, and ChildFriendliness (incl. a spaced form, e.g. "very
suitable").
```

The repository performs no text filtering; it simply returns the user's tours (with logs). Because `Popularity` and `ChildFriendliness` are computed C# properties rather than database columns, the search predicate cannot run in SQL — so the BL evaluates it in memory. The frontend sends the query as `GET /api/v1/tours?q=<text>` only after the debounce delay elapses.

Debounce implementation

The raw input value and the debounced value are kept as two separate state variables in `useTourListViewModel`. A `useEffect` resets a 300 ms `setTimeout` on every keystroke; only when the user stops typing does `debouncedSearch` update, which changes the TanStack Query key and triggers the actual API call. Without this, every individual keystroke would fire a separate `GET /api/v1/tours?q=…` request, causing unnecessary backend load and redundant in-flight requests that could return out of order.

```
// useTourListViewModel.ts
const [search, setSearch] = useState('')          // bound to input — updates on
every keystroke
const [debouncedSearch, setDebouncedSearch] = useState('')  // drives the query key

useEffect(() => {
  const timer = setTimeout(() => setDebouncedSearch(search), 300)
  return () => clearTimeout(timer)   // cancel if user types again before 300 ms
}, [search])

useQuery({
  queryKey: [TOURS_QUERY_KEY, debouncedSearch],  // API only called after 300 ms
idle
  queryFn:  () => toursApi.getAll(debouncedSearch || undefined),
})
```

5 Unit Tests – Rationale and Coverage

36 NUnit tests in `TourPlanner.Tests` target the Business Logic layer. Moq replaces the repository and ORS dependencies; no database or HTTP stack is required.

> **Why BL is the critical test target:** The BL layer is the sole enforcer of ownership scoping, uniqueness invariants, and domain classification rules. Controllers are thin adapters; the DAL is a direct EF Core translation. A regression here directly breaks security (cross-user data access), data integrity (duplicate tour names), or correctness (wrong computed attributes shown to users).

5.1 TourServiceTests — 15 tests

`TourService` is the most-used BL component: it enforces per-user ownership, ORS route enrichment, and the full-text search (including computed attributes) on top of CRUD.

Test name	Why it is critical
`GetAllAsync_ReturnsToursForUser`	User-scoping must never leak another user's tours.
`GetAllAsync_SearchMatchesLogComment`	Full-text search must match text inside a tour's log comments.
`GetAllAsync_SearchMatchesComputedChildFriendliness`	Search must match the computed ChildFriendliness (e.g. "very suitable").
`GetAllAsync_SearchMatchesComputedPopularity`	Search must match the computed Popularity (log count).
`GetAllAsync_SearchNoMatch_ReturnsEmpty`	A non-matching query must return no tours.
`GetByIdAsync_ExistingTour_ReturnsTour`	Happy-path read for an owned tour confirms ownership check does not over-block.
`GetByIdAsync_NotFound_ThrowsTourNotFoundException`	Typed domain exception is required for 404 HTTP mapping; returning null would crash the controller.
`CreateAsync_DuplicateName_ThrowsAndDoesNotPersist`	The per-user unique-name rule must reject before any ORS call or write — no duplicate is saved.
`CreateAsync_PersistsOrsComputedRouteData`	Create must store ORS distance (km), time and route geometry + elevation.
`CreateAsync_OrsFailure_StillCreatesTourWithoutRoute`	A transient ORS failure must not prevent the tour from being created.
`UpdateAsync_ExistingTour_UpdatesFields`	All changed fields must be written; a missing field is a silent data-loss bug.
`UpdateAsync_RouteInputsUnchanged_DoesNotRecomputeRoute`	ORS must not be called when from/to/transport are unchanged (avoids waste).
`UpdateAsync_DestinationChanged_RecomputesRoute`	Changing the destination must trigger a fresh ORS route.
`UpdateAsync_NotFound_ThrowsTourNotFoundException`	Update of a missing tour must fail cleanly, not silently succeed on nothing.
`DeleteAsync_CallsRepository`	Delete must call DAL with the correct tour ID and user ID.

5.2 TourLogServiceTests — 10 tests

`TourLogService` validates the parent tour exists and belongs to the user before any log operation.

Test name	Why it is critical
`GetByTourIdAsync_ExistingTour_ReturnsLogs`	Happy path — logs returned when parent tour exists and is owned by the user.
`GetByTourIdAsync_TourNotFound_ThrowsTourNotFoundException`	Cannot list logs for a tour the user does not own — prevents information disclosure.
`GetByIdAsync_ExistingLog_ReturnsLog`	Single-log fetch by ID returns the correct entity.
`GetByIdAsync_NotFound_ThrowsTourLogNotFoundException`	Typed exception required for clean 404 mapping.
`CreateAsync_TourExists_CreatesLog`	Log creation succeeds only when parent tour is found and owned by user.
`CreateAsync_TourNotFound_ThrowsTourNotFoundException`	Prevents orphaned logs referencing non-existent or foreign tours.
`CreateAsync_DuplicateDateTime_ThrowsAndDoesNotPersist`	A second log for the same tour at the same time must be rejected, not written.
`UpdateAsync_ExistingLog_UpdatesFields`	All fields (distance, comment, difficulty, rating) must be updated correctly.
`UpdateAsync_NotFound_ThrowsTourLogNotFoundException`	Update of missing log must fail with the typed exception.
`DeleteAsync_CallsRepository`	Delete delegates to DAL with the correct IDs.

5.3 TourComputedAttributeTests — 6 tests

Pure C# property tests — no mocks required. These run entirely in-memory and cover the classification boundaries.

Test name	Why it is critical
`Popularity_NoLogs_ReturnsZero`	Zero logs must yield Popularity = 0, not throw or return a negative value.
`Popularity_WithLogs_ReturnsLogCount`	Regression would display a wrong count to every user viewing the tour.
`ChildFriendliness_NoLogs_ReturnsNotSuitable`	Safe default — must never show a positive result without supporting evidence.
`ChildFriendliness_AllEasyShortDistance_ReturnsVerySuitable`	Upper boundary: all Easy + avg distance ≤ 10 km must classify as VerySuitable.
`ChildFriendliness_MediumDistanceUnder30_ReturnsSuitable`	Middle boundary: Medium difficulty + distance ≤ 30 km must classify as Suitable.
`ChildFriendliness_HardLongDistance_ReturnsNotSuitable`	Hard + long distance must classify as NotSuitable — the most safety-sensitive outcome.

5.4 ImportExportServiceTests — 5 tests

Import/export writes tours and logs in bulk under the current user, so ownership and collision handling must be correct.

Test name	Why it is critical
`ImportAsync_AssignsOwnershipNewIdsAndCreatesLogs`	Imported tours/logs must get fresh ids and the importing user's ownership.
`ImportAsync_EmptyList_ThrowsBusinessLogicException`	An empty import must be rejected with a typed domain exception.
`ImportAsync_MissingRequiredFields_ThrowsBusinessLogicException`	Tours without name/from/to must be rejected, not persisted half-formed.
`ImportAsync_DuplicateName_AppendsCounterSuffix`	Name collisions must be resolved by appending " (n)" so import never fails on duplicates.
`ExportAsync_ReturnsUserTours`	Export must return exactly the requesting user's tours.

6 Time Tracking

Development spanned the full semester (February 3 – June 21, 2026). Backend work was led by Ricky Raveanu; frontend work by Nikola Petrovic. Time estimates are based on commit timestamps; the git history provides the full granular log.

Task / Phase	Ricky	Nikola	Commits / dates (approx.)
Monorepo init, Docker Compose dev + prod, CI workflows	10 h	—	Feb 3–4
.NET solution scaffold, layers, health endpoint, middleware	6 h	—	Feb 4
EF Core + PostgreSQL, migrations, DbContext	5 h	—	Feb 10
ASP.NET Identity, JWT auth, refresh tokens	6 h	—	Feb 11
Tour + TourLog domain models with computed attributes	4 h	—	Feb 12
DAL repositories (ITourRepository, ITourLogRepository, ...)	5 h	—	Feb 17–18
BL services (TourService, TourLogService)	5 h	—	Feb 18
API controllers (Tours, TourLogs) + FluentValidation + DTOs	5 h	—	Feb 25
ImageService + photo upload endpoint	3 h	—	Feb 26
Dev seed data (DataSeeder)	2 h	—	Mar 3
NUnit test suite (Moq, grown to 36 tests)	6 h	—	Mar 4 onward
ORS backend proxy (MapController + OrsService)	4 h	—	Apr 2
React frontend scaffold (Vite, TanStack Router, Tailwind)	—	8 h	Mar 10
Auth-guarded routing, app layout, Zustand auth store	—	5 h	Mar 10–11
Login + Register views with MVVM hooks	—	5 h	Mar 17
DataTable component, Tour model + API client	—	4 h	Mar 18
Tour list view with search, create, edit, delete	—	6 h	Mar 19

Task / Phase	Ricky	Nikola	Commits / dates (approx.)
Tour Detail view, computed attribute display, TourMap (Leaflet)	—	6 h	Mar 24
TourLog model + API client + CRUD views	—	6 h	Mar 25–26
Image upload field in tour form + display in detail view	—	3 h	Apr 1
UI wireframes	—	2 h	Apr 15
Unique feature: elevation profile (ORS elevation + Recharts)	2 h	4 h	May
ORS route persistence on create + startup backfill	5 h	2 h	Jun
Import / export of tour data (JSON)	3 h	2 h	Jun
Full-text search incl. computed attributes (BL)	3 h	1 h	Jun
Deployment / Coolify debugging (stale images)	4 h	—	Jun
Final protocol, UML updates, regeneration	4 h	3 h	Jun
TOTAL	**82 h**	**57 h**	**~139 combined hours over the semester**

6.1 Unique Feature — Elevation Profile Chart

The project's unique feature is an interactive elevation profile chart rendered directly in the Tour Detail view. When ORS returns route geometry, the backend fetches elevation data for each coordinate point and returns an elevation array alongside the route. The frontend renders this as a line chart using **Recharts**, showing altitude (metres) against distance (km) along the route.

- **Data source:** ORS elevation API — elevation values are sampled per route waypoint.
- **Rendering:** Recharts `<AreaChart>` with a gradient fill, responsive container, and custom tooltip showing altitude at each point.
- **Integration:** chart updates automatically when a new route is calculated; hidden when no route data is available yet.